How Panlith + Basalt work (PanVK in Zalith)

A technical document so anyone can pick up the project without prior context. I mark what is **confirmed** (seen in logs or in the code) and what is a **hypothesis**.

Test device: Samsung Galaxy A16 5G (SM-A166M), Exynos 1330, Mali-G68 MC2, Android 16.

---

1. Starting point

- Minecraft 26.x wants Vulkan. With the **system** Vulkan driver of this Mali, the game rejects the device because some extensions and features are missing: `VK_EXT_vertex_attribute_divisor`, `VK_KHR_push_descriptor`, `fillModeNonSolid`, `vertexAttributeInstanceRateDivisor` and `multiDrawIndirect`. Result: the game falls back to OpenGL (MobileGlues). *(confirmed in logs)*
- PanVK (Mesa/Panfrost with the kbase backend, by zenithblue-oss) does expose them. The log shows: `Using graphics backend Vulkan, using drivers: 1.3.363 panvk PanVK-kbase RC3-lowmem`.
- Project rules: no root, no bootloader unlock, no replacing or modifying the system Vulkan driver, no assumption of Adreno/Snapdragon, and a working fallback to OpenGL.

2. The pieces

Piece	What it is
`libvulkan_panfrost.so`	The PanVK-kbase **ICD** (binary by zenithblue-oss, built for Bionic/Android).
`libvulkan_panvk_shim.so`	Our own **shim** (a single C file). Sits between whoever asks for Vulkan and the ICD.
Basalt Driver	Plugin APK that carries both libraries and announces itself to the launcher.
Panlith Launcher	Fork of Zalith Launcher 2 with the patch that loads that driver.

3. Why a shim is needed

As shipped, the ICD exports **only** three symbols:

- `vk_icdGetInstanceProcAddr`
- `vk_icdGetPhysicalDeviceProcAddr`
- `vk_icdNegotiateLoaderICDInterfaceVersion`

That is the interface of an ICD meant for the system *Vulkan loader*. But Zalith/Pojav, LWJGL and Zink **do not go through the system loader** when a custom driver is used: they `dlopen` the library and look up `vkGetInstanceProcAddr` and `vkGetDeviceProcAddr` by name. The ICD does not export them, so something that does is required.

The shim:

- Exports **only** `vkGetInstanceProcAddr` and `vkGetDeviceProcAddr` (`-fvisibility=hidden` so nothing else leaks).
- Loads the ICD, negotiates interface v5 and forwards every other call.

- Implements `vkCreateAndroidSurfaceKHR` and `vkDestroySurfaceKHR` (with `VkIcdSurfaceAndroid`, platform 5), since the system loader is not there to do it.
- Adjusts the instance extension list (`VK_KHR_android_surface`) and returns empty layer lists.

4. How it is linked at runtime

1. **Plugin.** Basalt's APK declares `fclPlugin=true`, `driver=<name>` and `panvkDriver=true` in its manifest. It has `extractNativeLibs=true`, so both `.so` files are extracted into its native library directory (`nativeLibraryDir`).
2. **Discovery.** `DriverPluginManager` reads that metadata and creates a `Driver` whose `path` is that directory. The `isPanVK` flag comes from `panvkDriver=true` (our patch).
3. **Environment.** When launching the game, `GameLauncher.initEnv` sets `DRIVER_PATH`, `PANVK_SHIM_PATH`, `PANVK_ICD_PATH`, `MESA_SHADER_CACHE_DIR` and `MESA_GLSL_CACHE_DIR`.
4. **Native loading.** In `egl_bridge.c`, `load_vulkan()` first calls `loadPanVKVulkan()` (API ≥ 28): it `dlopen`s the shim, the shim negotiates with the ICD, and the result is published with `set_vulkan_ptr()` in the environment variable `VULKAN_PTR` (the `dlopen` handle, in hexadecimal).
5. **Who uses it.** Everything asks for `VULKAN_PTR`:
   - **LWJGL** (`getVulkanDriverHandle`) → Minecraft 26.x with native Vulkan.
   - **SDL hook** and `lwjgl_dlopen_hook.c` (replaces `libvulkan.so` with the handle).
   - **Zink** (OpenGL over Vulkan): Pojav's Mesa patch in `zink_screen.c` uses `strtoul(getenv("VULKAN_PTR"))` as the library and runs `dlsym` for `vkGetInstanceProcAddr` and `vkGetDeviceProcAddr`. If either is missing, Mesa writes `ZINK: failed to get proc address` to logcat (not stderr) and fails.
6. **Vulkan checker.** `VulkanCheckerViewModel` passes the shim path and `vulkan_checker.c` has a PanVK branch, so the initial check does not reject the driver.

Paths:

```
Minecraft 26.x : Minecraft -> LWJGL Vulkan -> shim -> PanVK ICD -> Mali
OpenGL (Zink)  : Minecraft -> OpenGL -> Kopper/Zink -> shim -> PanVK ICD -> Mali
Without PanVK  : Minecraft -> OpenGL -> MobileGlues (fallback)
```

The renderer `opengles3_desktopgl_zink_kopper` calls `load_vulkan()` and then sets `GALLIUM_DRIVER=zink` and `MESA_ANDROID_NO_KMS_SWRAST=1`.

Turnip: its path (`loadTurnipVulkan`, `linker_ns_*`) is only compiled under `ADRENO_POSSIBLE` and checks `checkAdrenoGraphics`, so it **never runs on Mali**. `POJAV_ZINK_PREFER_SYSTEM_DRIVER` makes the launcher skip any custom driver.

5. How to tell it works (in `latest_game.log`)

Good lines:

- `EGLBridge: LWJGL-side Vulkan loader requested the Vulkan handle`
- `PanVK: shim listo, handle: 0x...` and `VULKAN_PTR = 0x...`
- `WARNING: panvk is not a conformant Vulkan implementation, testing use only.`
- `Using graphics backend Vulkan, using drivers: ... panvk PanVK-kbase ...`
- `Using graphics device: Mali-G68 MC2 (ARM)`
- `DRIVER_PATH` pointing to the plugin package.

Lines that **look** bad but are not:

- `MESA-LOADER: failed to retrieve device information` also appears in logs that work.

- `Couldn't leave fullscreen`, `Failed to set window icon`: SDL on Android.
- Realms and narrator (flite) errors: local account / Android.
- `linker: library "libvndksupport.so" ... is not accessible for the namespace`: emitted by the ICD and does not prevent it from working.
- "Client shutdown from post-main" on exit: happens with PanVK, OpenGL and Zink; it is launcher/26.2 behavior.

6. Shim diagnostics

Two variants of the same code (`PANVK_SHIM_DIAG`):

- **Clean (0, the one that gets published):** no wrappers, no stderr, no tracing; only errors and one load line to logcat saying `build=clean`.
- **Diagnostic (1):** logs each requested name once (`TRACE gipa/gdpa`), the results of `vkCreateInstance`, `vkEnumeratePhysicalDevices`, `vkCreateDevice` and `vkCreateSwapchainKHR`, and every second a `PROF` line with FPS, frame time and, per call, `n/avg/max/sum` for `submit`, `present`, `acquire`, `waitFences`, `waitSem`, `queueWaitIdle` and `deviceWaitIdle`. Variables: `PANVK_SHIM_QUIET=1`, `PANVK_SHIM_TRACE=0`, `PANVK_SHIM_PROF=0`.

If the log shows `build=diag` or `TRACE` / `PROF` lines, the APK carries the diagnostic variant.

7. What was found along the way

- Before shim v4, Kopper/Zink failed with `GLFW: Failed to create window context!`. Shim v4 added the `vkGetDeviceProcAddr` export and Zink started working. **Probable cause, not isolated:** v4 changed several things at once.
- Performance: in the runs with diagnostics, `submit` takes ~1 ms and `present` ~30–45 ms (≈ 80 % of the frame). **Unconfirmed hypothesis:** `present` waits for the GPU to finish (GPU work or a WSI copy). Proposed test: an optional wait after each `submit` to measure GPU time.
- At 100 % resolution we see ~18–25 FPS; at 25 %, ~30–55. It does not scale 4× when pixels drop, which suggests a limit that does not depend on resolution (inference).
- Another user (zenithblue-oss issue #23, Mali-G615) reports a similar ceiling (~20 FPS) with the GPU at 100 %.
- OpenGL with MobileGlues performs noticeably better in the team's tests.

8. How each piece is built

- **Shim, on Windows:** NDK 27.3.13750724, SDK at `C:\Android`, CMake 3.31.6. `tools\build.ps1` (diagnostic) and `tools\build_clean.ps1` (clean) use the SDK's cmake and ninja. PowerShell breaks on multi-line pastes: use single-line commands joined with `;`.
- **adb:** exit code `-1073741515` (0xC0000135) is a missing DLL; put `C:\Android\platform-tools` first in PATH.
- **Plugin:** the workflow builds the clean shim with GitHub's NDK and assembles the APK. The ICD goes in `app/src/main/jniLibs/arm64-v8a/`.
- **Launcher:** JDK 21, Gradle 9.5, AGP 9.3, NDK 25.2.9519653. The **Panlith release arm64** workflow runs `assembleRelease -Darch=arm64`.
- **Patch order:** first `zalith_panvk_v3.patch` (PanVK support), then `panlith_fork_v2.patch` (name, id, icons, notice, updater disabled, signing, workflow).

9. Known pitfalls

- Every GitHub build signs with a different key: **uninstall the previous plugin** before installing the new one.
- Changing the driver name in the plugin forces you to **select it again** in the Minecraft version.
- Editing the plugin APK with MT Manager: replace the `.so` in `lib/arm64-v8a/` and sign V1+V2+V3.

- The launcher **debug** build is much slower than **release** (no R8, debuggable). Always compare release with release.
- FPS tests must use the same world, the same spot and the same scale. The numbers in this document are not controlled.
- Without own keys there are no CurseForge downloads and no Microsoft login. Modrinth works without a key.

10. Licenses

- Panlith Launcher: GPL-3.0, with Zalith's additional terms (different name, visible "Unofficial Modified Version", copyright notices intact).
- Basalt and shim own code: Apache-2.0.
- PanVK-kbase ICD: Mesa MIT/X11, ARM headers GPL-2.0 with syscall exception, and MIT scripts. **Redistribution of the compiled binary has not been settled**: see the pending issue to zenithblue-oss.

11. Credits

Zalith Launcher 2 and PojavLauncher · PanVK-kbase (zenithblue-oss), Mesa and Panfrost · PanProbe · PanVKDriverPlugin (muhibiscr1t1kal) · MobileGlues · Kopper-Zink (whaltermc) · IronizedZink · FCL Turnip plugin.

Made by one person and several AIs: ChatGPT (main ideas), Claude (code and builds), Grok (deep research) and Copilot (support).